

[L0-L1 Core Essence & Design Logic]

L0 Essence: : Under the physical limits of random writes on non-volatile media, RocksDB converts sparse random writes into sequential writes using an in-memory lock-free skip list paired with a write-ahead log (WAL) and background 级联 merge compactions, constructing an embedded key-value engine with extreme throughput and snapshot-isolated concurrency.

L1 Logic:

- :
- 1. Lock-Free SkipList & Append Writes** (M1 Write) leverage sequential WAL logging for durability, employing CAS optimistic synchronization and Arena allocators to eliminate lock contention and fragmentation.
 - 2. Compact SSTable Blocks & Prefix Queries** (M2 SSTable) reduce physical storage overhead through restart points for delta encoding, utilizing Bloom Filters to filter invalid random disk reads.
 - 3. Leveled Compaction & Amplification Trade-offs** (M3 Compaction) recycle dead space and enforce global key order via background merges, executing a zero-sum game between write, space, and read amplifications.
 - 4. Lockless Batches & Sequence Snapshots** (M5 Transaction) isolate locks by constructing lightweight MVCC based on global sequence numbers, supporting atomic Group Commit and 2PC coordination.

[L2 Storage Engine Data Flow Topology]

:

• [User WriteBatch] → [Sequential WAL Append] → [Concurrent MemTable Write] → [Frozen Immutable MemTable] → [Background Flush] → [L0 SSTable Files] → [Background Multi-Threaded Leveled Compaction] → [Uncompressed Block Cache] → [Row Cache] → [Snapshot Read]

[M1.1] Write Flow & WAL

- **Physical Path:** Write requests (Put/Delete) are first appended sequentially to the Write-Ahead Log (WAL) on disk. The mutations are then applied to the in-memory **MemTable**. Once both steps succeed, the write returns.
- **Sync Option:** Control durability via WriteOptions.sync. When true, every write forces an fsync system call to flush dirty page cache blocks to disk before returning, protecting against data loss.

[M1.2] SkipList MemTable

- **SkipList Structure:** The default MemTable uses an in-memory lock-free concurrent SkipList. Its multi-level index pointers provide $O(\log N)$ search and range scans.
- **Lock-Free CAS:** Pointer adjustments in the SkipList use Compare-And-Swap (CAS) optimistic concurrency, eliminating global mutex contention. It uses an Arena allocator to request large memory blocks from the OS, managing allocation internally to prevent standard heap fragmentation.

[M1.3] MemTable Selection

- **Types & Use Cases:**
 - SkipListMemTab (Default): Supports concurrent writes and range scans;
 - VectorMemTab: Uses std::vector, lack concurrent writes. Best for single-threaded bulk loading where queries search sorted vectors;
 - HashSkipList / HashLinkList: Adds hash buckets over a SkipList. Speeds up point lookups and prefix scans for keys with fixed prefixes.

[M1.4] Immutable MemTable & Flush

- **State Freeze:** When a MemTable reaches write_buffer_size, it is frozen into an **Immutable MemTable** (read-only), and a new active MemTable is spawned to accept incoming writes.
- **Flush Thread:** A background thread is scheduled to write the Immutable MemTable sequentially to a Level 0 (L0) SSTable file on disk. Once written, the corresponding WAL space is reclaimed.

[M1.5] Write Stall

- **Self-Preservation:** When background flushes or compactions lag behind client writes, RocksDB slows down or pauses writes to prevent memory exhaustion and disk saturation.
- **Trigger Conditions:**
 - L0 SSTable count hits level0_slowdown_writes_trigger (default 20, slows down writes);
 - Hitting level0_stop_writes_trigger (default 30, halts writes completely);
 - Pending compaction bytes exceed limits.

[M2.1] SSTable Format

- **Physical Structure:** SSTables use a Block-Based Table format. Keys are grouped into fixed-size **Data Blocks** (default 4KB). The file ends with metadata blocks: **Filter Block** (Bloom filters), **Index Block**, **MetaIndex Block**, and a 48-byte **Footer**.
- **Read Path:** Readers first fetch the Footer to locate the MetaIndex, which maps to Index and Filter blocks, pointing to the exact Data Block offsets and avoiding scanning.

[M2.2] Block Format & Restart Points

- **Prefix Sharing:** Keys inside a block are sorted. To save space, keys share prefixes with their predecessor, storing only shared_bytes/unshared_bytes/value.
- **Restart Points:** To search inside a block without scanning from the start, prefix compression is broken every 16 keys, creating a **Restart Point** with a full key. Offset arrays of these points are stored at the block footer for binary searching.

[M2.3] Bloom Filters

- **Lookup Bypass:** Bloom filters map keys to bit arrays using hash functions. With $O(1)$ operations, they verify if a key is "absolutely not present" before reading disk blocks.
- **Filter Formats:**
 - Block-based Filter (Legacy): Generates filter bits per 2KB data block, requiring multiple indexes;
 - Full Filter (Modern): Generates one global filter per SSTable file, cutting lookup overhead to a single I/O.

[M2.4] Index Block

- **Block Boundaries:** The Index Block maps to each Data Block. Index keys store the shortest separator key representing the block boundary, pointing to the block's physical offset and size.
- **Hash Index:** Supports Binary Search (standard binary indexing) and Hash Search (prefix hash indexing utilizing prefix Bloom filters to skip binary searches).

[M3.1] Size-Tiered Compaction

- **Grouping:** Groups SSTables of similar sizes into tiers. When a tier hits its file threshold, background threads merge-sort them into a larger SSTable in the next tier.
- **Amplification:** Low write amplification but high space amplification. Merging large files requires disk space equal to the dataset size, risking disk-full deadlocks.

[M3.2] Leveled Compaction

- **Layered Design:** Splits disk space into Level 0 to Level Max. L0 files can overlap, while higher levels (L1-LMax) have sorted, non-overlapping keys. Levels scale in size by 10x (L1=256MB, L2=2.5GB).
- **Cascading Merge:** When a level exceeds its budget, a background thread selects an SSTable and merges it with overlapping files in the next level, purging deleted keys and tombstones. Extremely low space amplification but high write amplification (10-30x).

[M3.3] Compaction Scores

- **Prioritization:** RocksDB computes a **Compaction Score** per level to schedule merging:
 - For L0: $Score = L0_file_count / level0_file_num_compaction_trigger$;
 - For $L_n (n \geq 1)$: $Score = actual_level_size / level_size_limit$.
- **Execution:** The thread picks the level with the highest score and merges files with the longest lifespans or minimal next-level overlap to save I/O.

[M3.4] Amplification Law

- **LSM Trade-offs:**
- **Write Amplification (WA):** Bytes written to disk vs bytes written by the user. High WA degrades SSD lifespans and causes write stalls;
- **Space Amplification (SA):** Disk footprint vs logical data size. High SA wastes disk space;
- **Read Amplification (RA):** Disk bytes read vs logical bytes retrieved. High RA degrades read QPS.

[M3.5] FIFO Compaction

- **Time-Series Focus:** A fast compaction mode. Files exist only in L0. When total data size exceeds max_table_size_definition, the oldest SSTable is deleted.
- **Zero WA:** Write amplification is 1. No CPU-intensive merge-sorting occurs. Best for time-series metrics, logs, and expiring caches.

🔍 RocksDB Architectural Trade-offs & Fallacies Matrix

- Intuition 1:** Since RocksDB supports WriteBatch, we can safely pack hundreds of thousands of updates into a single batch to speed up loading.: Reality: WriteBatch must be fully parsed in memory to build MemTable nodes. Large batches trigger memory allocation thrashing (STW) and lock out other threads by holding write locks too long. → Split large batches. Limit WriteBatch size to 1,000 - 10,000 keys. For massive loading, use SST File Writer to build SST files off-target and Ingest them.
- Intuition 2:** The larger the Block Cache is, the higher the read throughput will be; we should allocate all free host memory to it.: Reality: LSM engines search indexes and filters of multiple L0 files concurrently. Starving MemTable memory triggers frequent flushes, and page cache starvation spikes random disk reads. → Limit Block Cache to 30% - 50% of physical RAM. Enable cache_index_and_filter_blocks and set up a global WriteBufferManager to control MemTable memory bounds.
- Intuition 3:** Leveled Compaction is fast and space-efficient; we should enforce it for all workloads.: Reality: Under heavy, sustained random writes, Leveled Compaction's 10x size ratio spikes write amplification (WA to 20-30x), saturation disk IO, causing write stalls and latency spikes. → For write-heavy time-series data (metrics, logs), choose FIFO Compaction or Universal Compaction to reduce write amplification to 1.5 - 2x.
- Intuition 4:** Setting sync=false accelerates writes while providing the same durability as sync=true during power loss.: Reality: sync=false writes WAL only to the OS page cache. A hard crash or power cut will lose un-flushed data, violating atomic transaction guarantees. → For strong durability, set sync=true in WriteOptions. For throughput balance, use Group Commit (automatically handled by RocksDB's WriteThread).
- Intuition 5:** We can retrieve keys concurrently across threads using Get() without configuring a Block Cache.: Reality: LSM trees have no global in-place index. Without Block Cache, every Get() searches multiple SST index blocks and triggers random disk I/O, spiking read latency. → Configure Block Cache and set pin_l0_filter_and_index_blocks_in_cache=true to force L0 file metadata to remain in RAM, preventing random read misses.

[M4.1] Block Cache

- **Data Block Cache:** Caches uncompressed Data Blocks retrieved from disk. RocksDB provides LRU**C**ache (least-recently-used via double-linked list and hash) and C**l**ockCache (circular pointer scan with higher concurrency).
- **Shard Locks:** To prevent global mutex lock contention under concurrent threads, Block Cache is sharded (default 16 shards). Keys map to different lock domains, reducing cache overhead.

[M4.2] Compressed Cache

- **Secondary Cache:** Positions between Block Cache and disk. It caches raw, compressed disk blocks directly in memory.
- **CPU Trade-off:**
 - Uncompressed Cache (Block Cache): Stores C++ objects, saving CPU decompression cycles but consuming more RAM;
 - Compressed Cache: Uses less RAM but requires CPU cycles to decompress blocks on cache hits. Recommended only when memory is scarce and CPU is idle.

[M4.3] Row Cache

- **Row-Level Cache:** Positions above Block Cache, caching logical Key-Value pairs rather than physical disk blocks.
- **Fast Path:** Read operations check Row Cache first. On hits, values return immediately, bypassing block index binary searches. On misses, it reads via Block Cache and populates Row Cache.

[M4.4] Pinning Index & Filters

- **Read Miss Shield:** Index and filter blocks are large. If evicted from Block Cache, point lookups require 2-3 random disk I/O reads to reload them.
- **RAM Retention:** Set `pin_l0_filter_and_index_blocks_in_cache=true` (or `pin_top_level_index_and_filter`) to lock L0 and top-level metadata in memory, preventing LRU eviction and stabilizing latency.

[M5.1] Snapshots & MVCC

- **Sequence Tracking:** RocksDB increments a global **Sequence Number** for every write. Creating a snapshot binds it to the current Sequence Number.
- **Visibility:** Read operations (`Get/Iterator`) using a snapshot only see records where `SequenceNumber_record <= SequenceNumber_snapshot`. Reads bypass locks, enabling high-performance MVCC.

[M5.2] WriteBatch & Group Commit

- **Request Aggregation:** Concurrent user writes queue up in a global `WriteThread`. Through Group Commit, the queue leader thread locks the path and writes logs on behalf of its followers.
- **Atomic Writing:** The leader writes the combined batch sequentially to the WAL and performs disk syncs. Followers then parallelize writing their records into their respective MemTables, reducing thread context switches.

[M5.3] Optimistic Transactions

- **Conflict Detection:** Write operations (`Put/Delete`) do not lock rows. The transaction tracks the initial `Read Snapshot SequenceNumber`.
- **Commit Validation:** At commit time, the transaction verifies if keys have been modified by checking for higher sequence numbers. If a conflict is found, it rolls back the batch. Best for low-contention environments.

[M5.4] Pessimistic Transactions

- **Row Locking:** Acquires row-level locks via a lock manager (`LockManager`) during writes or calls to `GetForUpdate`.
- **Deadlock Graph:** If a lock is blocked beyond a timeout, the `Deadlock Detector` builds a Wait-For Graph in memory. If a cycle is detected, it aborts the transaction with the lowest cost to break the deadlock.

[M5.5] Two-Phase Commit (2PC)

- **Distributed Consensus:** Enables RocksDB to act as an engine for distributed databases (e.g., TiDB, CockroachDB).
- **WAL State Log:** Phase 1 writes a `Prepare` marker and transaction data to the WAL, keeping it invisible. Phase 2 writes a `Commit` marker upon coordinator confirmation, releasing the state to snapshots.

[M6.1] Column Families

- **Logical DBs:** Partitions a single RocksDB instance into logical namespaces. They share a single WAL to ensure atomic crash recovery.
- **Fine-Grained Tuning:** Each Column Family manages its own MemTable, SSTables, Block Cache, and Compaction parameters, isolating resources between workloads.

[M6.2] SSTFileWriter

- **Bypassing Writes:** Avoids standard write paths (WAL, MemTable locks, compaction) during massive bulk imports.
- **Offline SST Gen:** Uses `SSTFileWriter` to write sorted SSTable files directly in memory and calls `IngestExternalFile` to link them to RocksDB folders. The database mounts them instantly via manifest edits.

[M6.3] Rate Limiter

- **I/O Protections:** Background compactions and flushes can saturate disk bandwidth, causing client read/write latency spikes.
- **Throttling:** Configure a `RateLimiter` to throttle the bytes written by `Flush` and `Compaction`, smoothing out disk throughput and securing stable client latencies.

[M6.4] Stats Dump

- **Internal Profiling:** RocksDB regularly dumps performance statistics to log files.
- **Key Metrics:** Analyze:
 - **Compaction Stats:** Read/write amplification multipliers and level I/O bandwidth;
 - **Cache Hit Rate:** Index/Data block hit rates in Block Cache;
 - **Stall Duration:** Write Stall durations to pinpoint write path blockages.

[M6.5] Manifest Recovery

- **Metadata Index:** The `MANIFEST` file tracks active SSTables and their levels (`VersionSet`) in the database directory.
- **Recovery:** If the `MANIFEST` is corrupted during power cuts, the engine cannot boot. Restore it by calling `RepairDB`, which scans SST file headers and rebuilds the version set metadata index.

🗺️ Zone T: RocksDB Performance & Diagnostic Labs

T1 RocksDB Tuning Parameters Reference

Memory & Write Buffers:
`write_buffer_size`: Size of active MemTable. Default 64MB (increase to 128MB-256MB for high write throughput).
`max_write_buffer_number`: Maximize Immutable MemTables. Recommended 4 - 6.
`max_background_jobs`: Background threads for Flush/Compaction. Set to host CPU core count.
Compaction Triggers:
`level0_file_num_compaction_trigger`: L0 files to trigger compaction. Recommended 4.
`level0_slowdown_writes_trigger`: L0 files to trigger write slowdown. Default 20.
`level0_stop_writes_trigger`: L0 files to trigger absolute write halt. Default 36.

T2 LSM Engine Tuning Gold Standards

Write Amplification (WA) Targets:
Time-Series (FIFO Compaction) → Write amplification target 1 - 1.5.
General Workloads (Leveled Compaction) → Target 10 - 25 under production writes.
RAM Allocation Ratios:
Ratios → 10% OS page cache | 30% RocksDB MemTable (via `WriteBufferManager`) | 60% Block Cache.
Ensure `cache_index_and_filter_blocks=true` in Block Cache to keep metadata resident.

T3 Troubleshooting RocksDB Bottlenecks

1. **Resolving Write Stalls**
• *Symptom:* Client write QPS drops to zero, logs show `Stalling writes due to L0 files...`
• *Tuning:*
1. Check disk I/O; if SSD bandwidth is saturated, configure `RateLimiter` to cap compaction writes;
2. Increase `max_background_jobs` to allocate more CPU cores to compaction;
3. Raise `level0_slowdown_writes_trigger` to 30 and `level0_stop_writes_trigger` to 45.
2. **Debugging Low Block Cache Hit Rates**
• *Symptom:* Point lookup (`Get`) latencies spike, Block Cache hit rate falls below 40%.
• *Tuning:*
1. Set `pin_l0_filter_and_index_blocks_in_cache=true` to lock L0 metadata;
2. Disable Row Cache if key queries are highly sparse to prevent memory eviction;
3. Shard Block Cache (e.g., `num_shard_bits=6`) to reduce lock contention under concurrent reads.
3. **Recovering from Corrupted MANIFESTs**
• *Symptom:* Startup fails with `Corrupted MANIFEST or SST file checksum mismatch`.
• *Recovery:*
1. Run `ldb diagnostics: ldb --db=<db_path> check_sst`;
2. For MANIFEST logic corruptions, backup the folder and call `RepairDB` API to rebuild metadata;
3. Enable `paranoid_checks=true` in production to catch early block corruptions.